Транзакции

Транзакция - это последовательность операций с базой данных, которая выполняется как единое целое. Транзакция либо завершается успешно (COMMIT), либо полностью отменяется (ROLLBACK), чтобы гарантировать целостность данных.

Свойства:

Atomicity (Атомарность) - Транзакция выполняется полностью или не выполняется совсем.

Consistency (Согласованность) - после завершения транзакции база данных остаётся в согласованном состоянии, то есть не возникает нарушений ограничений целостности.

Ограничение целостности

- NOT NULL - запрет NULL значений
- UNIQUE - уникальность значений.
- PRIMARY KEY - первичный ключ (NOT NULL + UNIQUE)
- FOREIGN KEY - внешний ключ (связь между таблицами)
- CHECK - проверка условий
- EXCLUSION CONSTRAINTS - расширеные ограничения

EXCUSION CONSTRAINTS

```sql
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE room_booking (
    booking_id bigserial PRIMARY KEY,
    room_id    int NOT NULL,
    start_time timestamp NOT NULL,
    end_time   timestamp NOT NULL,
    CHECK (end_time > start_time),
    EXCLUDE USING gist (
        room_id WITH =,
        (tsrange(start_time, end_time, '[)')) WITH &&
    )
);
```

Isolation (Изоляция) - Параллельные

транзакции выполняются так, как если бы они были последовательными, предотвращая конфликты данных.

Durability (Долговечность) – После фиксации (COMMIT) изменения остаются в базе данных, даже если система выйдет из строя.

Уровни изоляции транзакций

- Read Uncommitted – Позволяет читать даже незавершённые изменения других транзакций (Есть в стандарте SQL, но в PostgreSQL фактически работает как Read Committed)
- READ COMMITTED (по умолчанию) – Транзакция видит только зафиксированные (COMMIT) изменения других транзакций.
- REPEATABLE READ – Транзакция видит данные в том виде, в каком они были на момент её начала, даже если

другие транзакции вносят.

- SERIALIZABLE - максимальный уровень изоляции. Гарантирует, что параллельные транзакции выполняются так, как если бы они шли последовательно.

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Аномалии

- Потерянное обновление - Две транзакции читают одну и ту же строку таблицы, затем одна транзакция обновляет эту строку, а после этого вторая транзакция тоже обновляет ту же строку, не учитывая изменений, сделанных первой транзакцией.

Время	Транзакция A (Перевод 200 на ACC002)	Транзакция B (Перевод 300 на ACC003)	Баланс ACC001
t1	BEGIN;	-	1000.00
t2	SELECT balance FROM accounts WHERE account_number = 'ACC001'; -> 1000.00	-	1000.00
t3	-	BEGIN;	1000.00
t4	-	SELECT balance FROM accounts WHERE account_number = 'ACC001'; -> 1000.00	1000.00
t5	UPDATE accounts SET balance = balance - 200 WHERE account_number = 'ACC001';	-	800.00
t6	-	UPDATE accounts SET balance = balance - 300 WHERE account_number = 'ACC001';	800.00
t7	COMMIT;	-	800.00
t8	-	UPDATE разблокируется и выполняется: balance = 800 - 300;	500.00
t9	-	COMMIT;	500.00

- **Грязное чтение** - Транзакция читает ещё не зафиксированные изменения, сделанные другой транзакцией.

Время	Транзакция A (Обновление бюджетов)	Транзакция B (Формирование отчета)	Бюджет IT	Бюджет HR
t1	`BEGIN;`	-	100000	80000
t2	`UPDATE departments SET budget = 120000 WHERE name = 'IT';`	-	120000	80000
t3	`UPDATE departments SET budget = 60000 WHERE name = 'HR';`	-	120000	60000
t4	-	`SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED; BEGIN;`	120000	60000
t5	-	`SELECT SUM(budget) FROM departments; -> 180000`	120000	60000
t6	-	`COMMIT;`	120000	60000
t7	`ROLLBACK;`	-	100000	80000
t8	-	-	100000	80000

- **Неповторяющееся чтение** – Транзакция читает одну и ту же строку два раза, и в промежутке между чтениями вторая транзакция изменяет (или удаляет) эту строку и фиксирует изменения. Тогда первая транзакция получит разные результаты.

Анти – паттерн

```
IF (SELECT amount FROM accounts WHERE id = 1) >= 1000 THEN
  UPDATE accounts SET amount = amount - 1000 WHERE id = 1;
END IF;
```

- **Фантомное чтение** – Транзакция два читает набор строк по одному и тому же условию, и в промежутке между чтениями, вторая транзакция добавляет строки, удовлетворяющие этому условию (и фиксирует изменения). Тогда первая транзакция получит разные наборы строк.

Время	Транзакция A (Анализ данных)	Транзакция B (Очистка данных)	Запись "Мышь"
t1	`BEGIN;`	-	Существует
t2	`SELECT COUNT(*) FROM products; -> 3`	-	Существует
t3	-	`BEGIN; DELETE FROM products WHERE name = 'Мышь'; COMMIT;`	Удалена
t4	`SELECT COUNT(*) FROM products; -> 2`	-	Удалена
t5	`SELECT * FROM products WHERE quantity > 5; -> ('Мышь', 2500, 10)`	-	Удалена
t6	`COMMIT;`	-	Удалена

Serializable

Уровень Serializable должен предотвращать вообще все аномалии. Это означает, что на таком уровне разработчику приложения не надо думать об одновременном выполнении. Если транзакции выполняют корректные последовательности операторов, работая в одиночку, данные будут согласованы и при одновременной работе этих транзакций.

Аномалия сериализации

- Циклическая зависимость данных

(Write Skew)

Время	Транзакция 1	Транзакция 2	SUM(category='a')	SUM(category='b')
t1	BEGIN;	-	30	15
t2	SELECT SUM(value) FROM... category='a' -> 30	-	30	15
t3	-	BEGIN;	30	15
t4	-	SELECT SUM(value) FROM... category='b' -> 15	30	15
t5	INSERT INTO ... (5,'b',30)	-	30	15+30=45
t6	-	INSERT INTO ... (6,'a',15)	30+15=45	45
t7	COMMIT;	-	45	45
t8	-	COMMIT; (ОШИБКА СЕРИАЛИЗАЦИИ)	45	45

Почему именно такие уровни в стандарте

– Разница между уровнями изоляции стандарта объясняется как раз кол-ом необходимых блокировок.

– Если транзакция блокирует изменяемые строки от изменения, но не блокирует чтения, получаем уровень Read Uncommitted.

– Если транзакция блокирует изменяемые строки и от чтения (короткая), и от изменения, получаем уровень Read

Committed.

- Берём длинную блокировку на чтение и получаем Repeatable Read

- С Serializable проблема: невозможно заблокировать строку, которой ещё нет.

Из-за этого остаётся возможность фантомного чтения. Поэтому для реализации уровня Serializable обычных блокировок не хватает – нужно блокировать не строки, а а условия (предикаты). Такие блокировки и были названы предикатными.

Предзакатные блокировки.

Предзакатная блокировка – блокировка записей, удовлетворяющих определённому условию (предикату)

- Классическая теория, связывающая уровни изоляции с блокировками, была

заложена в работах Джима Грея.

– Предикатные блокировки почти не используют в реальных СУБД

– Проверка конфликта предикатов – вычислительно сложная задача

Упрощенная модель

Вместо предикатных блокировок на практике используют более простые механизмы:

– Блокировки диапазонов ключей: Наиболее распространённая практика. При сканировании диапазона транзакция блокирует не только существующие записи, но и „промежутки" между ними, а также запись, следующую за диапазоном. Это предотвращает вставку новых записей внутрь заблокированного диапазона

– Блокировка индексов и страниц: Более

грубый – блокировка целых страниц индекса или даже таблиц, что гарантирует отсутствие фантомов, но ценой значительного диапазона.

PostgreSQL использует оптимистичный подход, основанный на Serializable Snapshot Isolation.

Write-Ahead Logging (WAL)

WAL – стратегия, при которой все изменения данных сначала записываются в журнал, и только потом применяются к основным файлам базы данных

Структура WAL

- Header (24 байта)
- Page ID (8 байт)
- Old Data (размер страницы)
- New Data (размер страницы
- Checksum (8 байт)

Ключевые параметры

- fsync=on
- synchronous_commit=on
- wal_buffers=64 MB

Synchronous_commit

- On - COMMIT дожидается, пока WAL гарантированно сброшен на диск (максимальная надёжность).
- Off - COMMIT возвращается сразу; данные могут быть потеряны при краже (но БД останется консистентной).
- local - ждём fsync только локально (без подтверждения реплик).
- remote_write - ждём, пока WAL дошёл до памяти standby-реплики.
- remote_apply - ждёт, пока WAL применён на standby (самый строгий режим)

Checkpoint

Checkpoint - это процесс, при котором все измененные данные из буферного кэша сбрасываются на диск, создавая согласованное состояние базы данных

Multi-Version Concurrency Control (MVCC)

MVCC - механизм многоверсионности, обеспечивающий параллельную работу транзакций без блокировки чтения записью и наоборот.

Принцип работы:

- UPDATE - создаёт новую версию строки, сохраняя старую.
- DELETE - помечает строку как удалённую
- INSERT - создаёт новую строку.
- Snapshot - определяет, какие версии строк видны транзакции.

Служебные поля

- xmin - ID транзакции, создавшей версию строки.

- xmax - ID транзакции, удалившей или заменившей версию.

Преимущества: – высокая параллельность
– чтение не блокирует версию

Недостаток: – накопление устаревших версий строк, которые очищаются с помощью VACUUM